```
package com.polarbookshop.catalogservice.domain;

import java.util.Set;
import javax.validation.ConstraintViolation;
import javax.validation.Validation;
import javax.validation.Validator;
import javax.validation.ValidatorFactory;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.Test;
import static org.assertj.core.api.Assertions.assertThat;

class BookValidationTests {
  private static Validator validator;

  @BeforeAll                                                    ❶
  static void setUp() {
    ValidatorFactory factory = Validation.buildDefaultValidatorFactory();
    validator = factory.getValidator();
  }

  @Test                                                         ❷
  void whenAllFieldsCorrectThenValidationSucceeds() {
    var book =                                                  ❸
      new Book("1234567890", "Title", "Author", 9.90);
    Set<ConstraintViolation<Book>> violations = validator.validate(book);
    assertThat(violations).isEmpty();                           ❹
  }

  @Test
  void whenIsbnDefinedButIncorrectThenValidationFails() {
    var book =                                                  ❺
      new Book("a234567890", "Title", "Author", 9.90);
    Set<ConstraintViolation<Book>> violations = validator.validate(book);
    assertThat(violations).hasSize(1);
    assertThat(violations.iterator().next().getMessage())
      .isEqualTo("The ISBN format must be valid.");             ❻
  }
}
```

❶ Identifies a block of code executed before all tests in the class

❷ Identifies a test case

❸ Creates a book with a valid ISBN

❹ Asserts that there is no validation error

❺ Creates a book with a non-valid ISBN code

❻ Asserts that the violated validation constraint is about the incorrect ISBN

Then we can run the tests with the following command: